CLM CONSULTANTS

WEBSITE ARCHITECTURE, COMPONENT DESIGN AND SECURITY DOCUMENTATION

A technical architecture blueprint for a corporate website and interactive web application

FRONTEND	BACKEND	DATABASE
Vue.js 3	Laravel	MySQL

Document type: Technical system architecture and reverse-engineering documentation
Scope: Frontend components, backend modules, MySQL database, API design, security, deployment and testing

Prepared as a proposed implementation architecture. Internal source code and infrastructure of the live website have not been independently verified.

DOCUMENT CONTROL

Item	Description
System	CLM Consultants website
Architecture style	Three-tier architecture with modular Laravel backend
Frontend	Vue.js 3
Backend	Laravel / PHP
Database	MySQL
API style	Versioned REST API
Security approach	Defense in depth; server-side authorization and validation
Document status	Proposed technical blueprint

IMPORTANT SCOPE NOTE

This document describes a proposed architecture using the technology choices specified: Laravel, Vue.js and **MySQL**. It is not proof that the live CLM Consultants website currently uses this exact stack or internal design. Confirm actual implementation details through authorized source-code review, deployment inspection and testing.

TABLE OF CONTENTS

- 1. System overview
- 2. Complete system architecture
- 3. Frontend component architecture
- 4. Laravel backend architecture
- 5. MySQL database architecture
- 6. REST API architecture
- 7. Security architecture
- 8. Authentication and authorization flows
- 9. Protected user workflows
- 10. Background processing
- 11. Deployment architecture
- 12. Testing and quality assurance
- 13. Maintenance and monitoring
- 14. Implementation roadmap
- 15. Conclusion

CHAPTER 1: SYSTEM OVERVIEW

1.1 Introduction

The CLM Consultants website is treated as a corporate digital platform for presenting company information, professional services, publications, consultation opportunities, contact channels and discussion forum functionality. The proposed solution separates presentation, application processing and data storage.

1.2 Objectives

• Present company information and professional services.

• Allow visitors to submit consultation and contact inquiries.

• Publish news, articles and frequently asked questions.

• Support member registration and forum participation.

• Provide authorized staff with content and inquiry management.

• Protect accounts, personal information, uploaded files and business records.

1.3 Technology stack

Layer	Technology	Responsibility
Frontend	Vue.js 3	Pages, reusable components, forms and interaction
Styling	CSS / Tailwind CSS	Responsive presentation
Routing	Vue Router	Client-side navigation
State	Pinia	Shared frontend state
Backend	Laravel	Business rules, validation, authentication and authorization
API	Laravel REST API	JSON communication
Database	MySQL	Relational data storage
Authentication	Laravel Sanctum	First-party SPA session authentication
Web server	Nginx + PHP-FPM	HTTPS termination and PHP request handling
Cache / Queue	Redis (optional)	Caching and background jobs
Storage	Laravel filesystem	Images, documents and backups

1.4 Architecture principles

• Separation of concerns: frontend, backend and database have distinct responsibilities.

• Least privilege: users and services receive only the access they require.

• Server-side trust: Laravel validates and authorizes every protected operation.

• Reusable components: common interface elements are shared across pages.

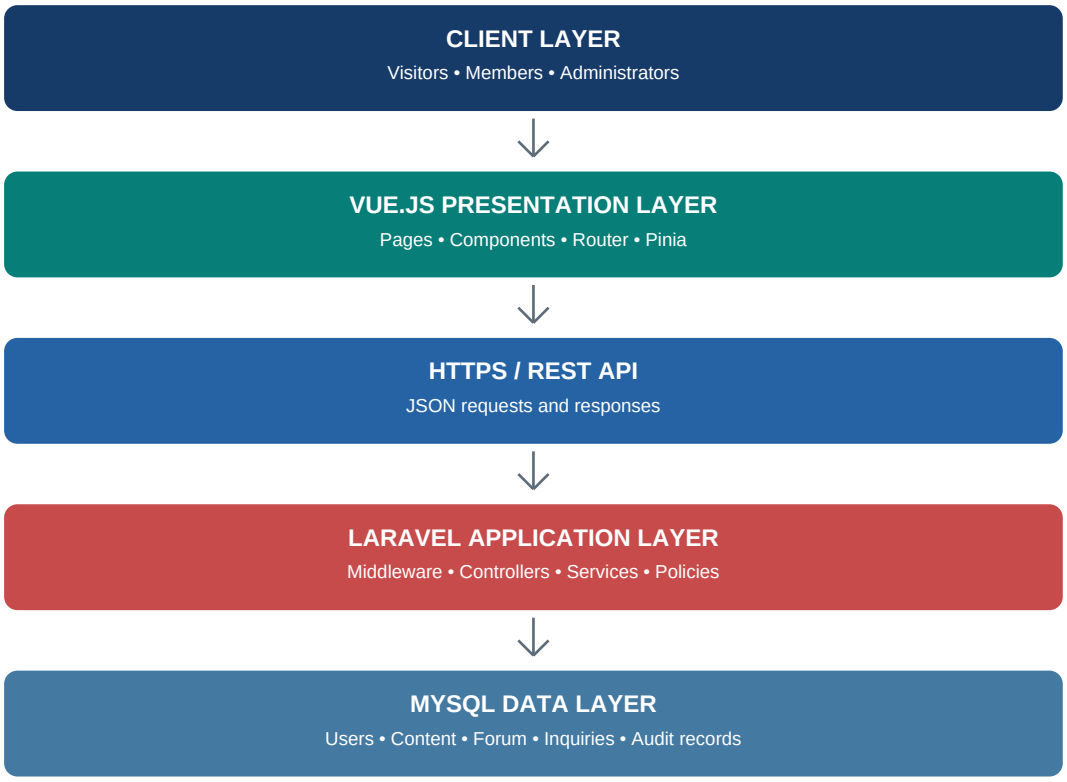
• Maintainability: business logic is organized into focused modules and services.

• Recoverability: backups, monitoring and tested restoration procedures are maintained.

CHAPTER 2: COMPLETE SYSTEM ARCHITECTURE

2.1 High-level architecture

The system uses a three-tier architecture supported by infrastructure services. The browser renders the Vue.js application; Laravel processes API requests and applies business rules; MySQL stores durable application records.



2.2 Request and response sequence

• A browser requests the website over HTTPS.

• Vue.js renders the relevant route and components.

• A user action triggers an API request where server data is required.

• Nginx forwards the request to the Laravel application through PHP-FPM.

• Laravel middleware applies request, session, authentication and security checks.

• Form Request classes validate input; policies authorize protected actions.

• Controllers call service classes, which use Eloquent models and MySQL transactions as needed.

• Laravel returns a JSON response; Vue.js updates the interface.

2.3 Trust boundaries

Treat the browser, submitted form values, uploaded files and all client-supplied identifiers as untrusted. The API boundary is a security boundary. The Laravel application is the authority for identity, permissions, validation and business rules. MySQL should not be publicly reachable.

CHAPTER 3: FRONTEND COMPONENT ARCHITECTURE

3.1 Recommended Vue.js project structure

```
frontend/
  public/
  src/
    assets/                 # images, icons, styles
    components/
      common/               # Header, Footer, Button, Input, Modal
      home/                 # Hero, Introduction, Services, News, FAQ
      about/
      services/
      news/
      consultation/
      contact/
      forum/                # CategoryList, TopicList, PostEditor
      admin/
    pages/                  # route-level page components
    layouts/                # MainLayout, AuthLayout, AdminLayout
    router/                 # Vue Router definitions
    stores/                 # Pinia auth, user and forum stores
    services/               # API client and domain API modules
    composables/
    utils/
    App.vue
    main.js
  package.json
```

3.2 Shared components

Component	Subcomponents / responsibilities
Header	Logo, navigation links, dropdowns, mobile menu, consultation CTA
Footer	Company summary, contact details, useful links, social links, copyright
Form controls	Text input, email, phone, select, checkbox, textarea
Feedback	Loading indicator, validation message, toast, empty state
Navigation	Breadcrumbs, pagination, tabs, mobile navigation
Content display	Cards, section heading, modal, accordion

3.3 Page and component hierarchy

```
App.vue
 ■■■ MainLayout
     ■■■ Header
     ■■■ RouterView
     ■   ■■■ Home
     ■   ■   ■■■ HeroSection
     ■   ■   ■■■ CompanyIntroduction
     ■   ■   ■■■ DiscussionSection
     ■   ■   ■■■ ServicesOverview
     ■   ■   ■■■ MELSection
     ■   ■   ■■■ NewsSection
     ■   ■   ■■■ FAQSection
     ■   ■   ■■■ ResearchSection
     ■   ■■■ About
     ■   ■■■ Services
     ■   ■■■ News / ArticleDetails
     ■   ■■■ Forum
     ■   ■■■ Consultation
     ■   ■■■ Contact
     ■   ■■■ Login / Register
     ■   ■■■ AdminDashboard
     ■■■ Footer
```

3.4 Frontend responsibilities

• Render responsive interfaces and reusable UI components.

- Perform client-side validation for usability, while repeating all important validation on the server.
- Manage navigation and display authenticated or role-specific views.
- Handle loading, empty, success and error states.
- Never embed database credentials, private API secrets or trusted authorization decisions in frontend code.

CHAPTER 4: LARAVEL BACKEND ARCHITECTURE

4.1 Backend organization

```
backend/
  app/
    Models/                # Eloquent entities
    Http/
      Controllers/         # HTTP request coordination
      Middleware/          # request and security filters
      Requests/            # server-side validation
      Resources/            # consistent JSON responses
    Services/              # business operations
    Policies/              # record-level authorization
    Jobs/                  # queued background work
    Mail/
    Notifications/
  database/
    migrations/
    seeders/
    factories/
  routes/
    api.php
    web.php
  storage/
  tests/
  config/
  .env                      # secrets; never commit
```

4.2 Functional backend modules

Module	Core responsibilities
Authentication	Register, login, logout, password reset, session lifecycle and verification
User management	User records, account status, roles and permissions
Content management	Company profile, services, news, articles, FAQs and media
Consultation	Receive, validate, store, assign and track consultation inquiries
Contact	Receive and manage visitor messages
Forum	Categories, topics, posts, member activity and moderation
Administration	Manage users, content, inquiries, roles and settings
Audit and monitoring	Record security-relevant administrative and account events

4.3 Laravel request processing

Routes should remain thin. Controllers coordinate HTTP input and output; Form Request classes validate; Policies or Gates authorize; Services implement business operations; Eloquent Models represent database entities. Use database transactions when an operation must update multiple related records atomically.

4.4 Example service-oriented flow

```
API Route
   -> Authentication / throttling middleware
   -> Controller
   -> Form Request validation
   -> Policy authorization
   -> Application Service
   -> Eloquent Model / MySQL transaction
   -> API Resource
   -> JSON response
```

CHAPTER 5: MYSQL DATABASE ARCHITECTURE

5.1 Database design principles

Use MySQL as the relational database. Design tables around business entities, define primary and foreign keys, apply unique constraints and indexes, and use transactions for multi-record operations. Store timestamps consistently and document retention rules for personal information.

5.2 Proposed tables

Domain	Tables	Purpose
Identity and access	users, roles, permissions, role_user, permission_role	Accounts and role-based access
Corporate content	company_profiles, services, categories, news, articles, faqs, media	Public website content
Consultation	consultations, consultation_services	Client requests and selected services
Contact	contact_messages	Visitor messages
Forum	forum_categories, forum_topics, forum_posts, forum_reports	Community discussions and moderation
Security	audit_logs, notifications, password_reset_tokens, sessions	Security records and application notifications

5.3 Key relationships

```
users 1 ---- many consultations
users 1 ---- many contact_messages (optional for guest messages)
users 1 ---- many forum_topics
users 1 ---- many forum_posts
forum_categories 1 ---- many forum_topics
forum_topics 1 ---- many forum_posts
services many ---- many consultations (through consultation_services)
roles many ---- many users (through role_user)
roles many ---- many permissions (through permission_role)
users 1 ---- many audit_logs
```

5.4 Important database controls

• Use InnoDB and foreign key constraints for relational integrity.

• Use unique indexes for fields such as normalized email addresses where appropriate.

• Index common lookup fields such as status, created_at, slug and foreign keys.

• Use parameterized queries or Eloquent; never concatenate untrusted input into SQL.

• Use transactions for related multi-table updates.

• Avoid storing plaintext passwords, authentication tokens or unnecessary sensitive data.

• Define soft-delete and retention rules deliberately; soft deletion is not a substitute for data-retention policy.

5.5 Example conceptual schema

```
users
  id (PK), name, email (UNIQUE), password,
  status, email_verified_at, created_at, updated_at

consultations
  id (PK), first_name, last_name, email, phone,
  message, status, assigned_to (FK -> users.id),
  created_at, updated_at

services
  id (PK), name, slug (UNIQUE), description, is_active

consultation_services
  id (PK), consultation_id (FK), service_id (FK)

forum_topics
  id (PK), category_id (FK), user_id (FK),
  title, body, status, created_at

forum_posts
  id (PK), topic_id (FK), user_id (FK),
  body, status, created_at
```

CHAPTER 6: REST API ARCHITECTURE

6.1 API conventions

Use a versioned API prefix such as /api/v1. Return JSON with consistent success and error structures, correct HTTP status codes, pagination metadata and validation errors. Do not expose internal exception traces in production.

6.2 Proposed endpoints

Method	Endpoint	Purpose / access
GET	/api/v1/services	Public service list
GET	/api/v1/services/{slug}	Public service detail
GET	/api/v1/news	Public published news
GET	/api/v1/news/{slug}	Public article detail
POST	/api/v1/consultations	Public, throttled inquiry submission
POST	/api/v1/contact-messages	Public, throttled contact submission
GET	/api/v1/forum/categories	Public forum categories
GET	/api/v1/forum/topics	Public topic list
POST	/api/v1/forum/topics	Authenticated member; permission checked
POST	/api/v1/forum/topics/{id}/posts	Authenticated member; permission checked
GET	/api/v1/auth/user	Authenticated current user
POST	/api/v1/auth/logout	Authenticated session logout
GET	/api/v1/admin/consultations	Authorized staff only
PATCH	/api/v1/admin/consultations/{id}	Authorized staff only

6.3 API response example

```
{
  "success": true,
  "message": "Consultation request received.",
  "data": {
    "reference": "generated-reference"
  }
}
```

Never return private inquiry details, password hashes, internal audit information or unrestricted user records through public endpoints.

CHAPTER 7: SECURITY ARCHITECTURE

7.1 Security objectives

Apply confidentiality, integrity and availability principles. Security must be layered: network, browser, application, identity, database, file storage, operations and monitoring.

7.2 Defense-in-depth controls

Layer	Threats / risks	Recommended controls
Network	Interception, exposed services	HTTPS/TLS, firewall, private database network, secure server configuration
Browser	XSS, unsafe scripts, leaked secrets	Vue escaping, avoid untrusted v-html, CSP, no secrets in bundle
Application	Injection, broken access control	Validation, Eloquent binding, policies, rate limits, safe error handling
Identity	Credential theft, brute force	Secure password hashing, throttling, MFA for privileged accounts, safe reset flow
Session	Session theft, CSRF	Secure HttpOnly cookies, SameSite, CSRF token, session regeneration
Database	Unauthorized access, data loss	Least-privilege DB user, private network, encrypted backups, patching
Files	Malicious uploads, data disclosure	MIME/content checks, random filenames, private storage, malware scanning
Operations	Undetected misuse, outages	Audit logs, alerts, backups, monitoring, incident response

7.3 Security headers

Configure headers appropriate to the application, including Content-Security-Policy, Strict-Transport-Security (after HTTPS is correctly deployed), X-Content-Type-Options: nosniff, Referrer-Policy and an appropriate frame-ancestors policy. Test CSP against required scripts, fonts and assets before enforcing it.

7.4 Secure configuration

- Set APP_DEBUG=false in production.
- Keep .env and private keys outside public access and source control.
- Use current supported Laravel, PHP, Vue and dependency versions.
- Restrict filesystem permissions and exposed server ports.
- Use separate least-privilege credentials for application and database services.
- Do not log passwords, session identifiers, reset tokens or authorization headers.
- Encrypt backups and restrict access to them.

CHAPTER 8: AUTHENTICATION AND AUTHORIZATION FLOWS

8.1 Recommended SPA authentication

For a first-party Vue SPA served from a configured trusted domain, use Laravel Sanctum's cookie-based SPA authentication. This approach uses the Laravel session and CSRF protection rather than storing long-lived bearer credentials in localStorage. Configure stateful domains, CORS credentials, cookie domain, Secure, HttpOnly and SameSite settings correctly for the deployment topology.

```
LOGIN FLOW

Browser / Vue
   -> Request CSRF cookie from Laravel Sanctum
   -> Submit credentials over HTTPS
Laravel
   -> Apply login throttling
   -> Validate credentials
   -> Check account status
   -> Regenerate session identifier
   -> Create authenticated session
Browser
   -> Store session cookie (HttpOnly)
   -> Send cookie with later requests
Laravel
   -> Validate session and CSRF on state-changing requests
   -> Return authorized response
```

8.2 Logout and recovery

Logout must invalidate the server-side session and regenerate the CSRF token. Password reset links must be random, time-limited, single-use and delivered to a verified destination. Avoid account-enumeration messages that reveal whether an email address is registered.

8.3 Role-based access control

Role	Example permissions
Super Administrator	System configuration, user and role administration
Administrator	Manage content, users and inquiries within assigned scope
Content Manager	Create, edit and publish assigned website content
Forum Moderator	Moderate topics, posts and reports
Registered Member	Create topics and replies subject to forum rules
Public Visitor	Read public content and submit public inquiries

8.4 Authorization decision flow

```
Incoming protected request
   -> Is the session authenticated?
      No  -> 401 Unauthorized
      Yes -> Load user and target resource
   -> Does the policy allow this action?
      No  -> 403 Forbidden
      Yes -> Execute authorized operation
   -> Record sensitive administrative action
   -> Return permitted response
```

Do not rely on hiding buttons or protecting Vue routes. Every protected backend operation must enforce authorization using Laravel policies, gates or equivalent server-side checks.

CHAPTER 9: PROTECTED USER WORKFLOWS

9.1 Consultation submission

```
Visitor
 -> Open consultation form
 -> Enter details and select service
 -> Vue checks required fields for usability
 -> Submit over HTTPS
Laravel
 -> Apply request throttling and CSRF protection
 -> Validate fields and service identifiers
 -> Apply spam controls
 -> Save inquiry in MySQL
 -> Dispatch notification job
 -> Return success and reference
Staff
 -> Authenticate and pass authorization policy
 -> Review inquiry and update status
```

Controls: length limits, format validation, rate limiting, anti-spam measures, safe output encoding, restricted inquiry access and retention rules.

9.2 Forum post workflow

```
Member submits topic or reply
 -> Laravel verifies session
 -> Validate request and content
 -> Check forum permission and category status
 -> Apply spam / abuse controls
 -> Save post with moderation status
 -> Notify relevant subscribers if enabled
 -> Moderator reviews reported content
 -> Record moderation action
```

Controls: authorization on every topic and post, output escaping, rate limits, moderation/reporting, content length limits and safe handling of links.

9.3 File upload workflow

```
Upload request
 -> Authenticate and authorize
 -> Enforce size limit
 -> Validate extension and detected MIME/content
 -> Generate random server-side filename
 -> Scan where available
 -> Store outside public web root or in private object storage
 -> Save metadata in MySQL
 -> Serve only through an authorized download route
```

Never trust a filename or Content-Type supplied by the browser. Do not allow executable uploads. Public image assets and private documents should have different access policies.

CHAPTER 10: BACKGROUND PROCESSING

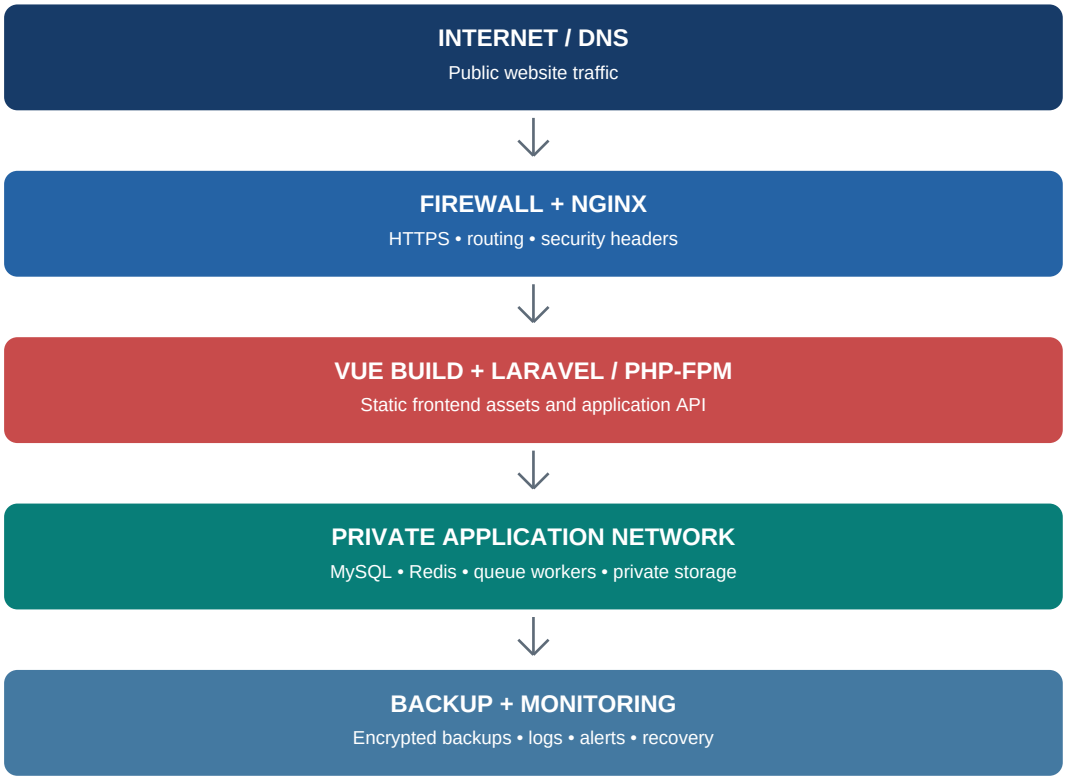
Use Laravel Queues for work that can safely happen after the visitor receives an initial response. Redis is an optional queue/cache service; it is not a replacement for MySQL as the durable relational database.

```
Form submitted
 -> Validate and store record in MySQL
 -> Dispatch Laravel job
 -> Queue backend (Redis or database queue)
 -> Queue worker processes job
 -> Send email / notification
 -> Record success or failure
 -> Retry according to bounded policy
```

Queue jobs should be idempotent where possible, have bounded retries, avoid logging secrets and send persistent failures to the failed-jobs mechanism for review.

CHAPTER 11: DEPLOYMENT ARCHITECTURE

11.1 Production topology



11.2 Deployment checklist

- Use a supported Linux server and supported PHP/Laravel versions.
- Build Vue assets for production; do not run the development server in production.
- Configure Nginx and PHP-FPM with least privilege.
- Enforce HTTPS and redirect HTTP to HTTPS.
- Keep MySQL bound to a private interface and allow only application access.
- Keep APP_DEBUG=false and protect environment secrets.
- Run migrations through a controlled deployment process and back up before risky schema changes.
- Run queue workers under a process supervisor.
- Set correct storage permissions and private upload handling.
- Configure automated encrypted backups and test restoration.

CHAPTER 12: TESTING AND QUALITY ASSURANCE

Test area	Examples
Frontend	Component tests, responsive layouts, navigation, form states and accessibility
Backend	Feature tests, validation, service logic, API response contracts
Database	Migrations, foreign keys, constraints, transaction rollback and query performance
Authentication	Login, logout, reset, session expiry, CSRF and throttling
Authorization	Role and resource ownership tests; verify 401 and 403 cases
Security	Injection, XSS, CSRF, IDOR, file upload, dependency and configuration checks
Performance	Page load, API latency, query plans, queue throughput and concurrency
Recovery	Backup integrity and restore drills

Security testing must be authorized and performed in a test environment wherever possible. Include negative tests proving that users cannot access records or actions outside their permissions.

CHAPTER 13: MAINTENANCE AND MONITORING

Operational monitoring should cover application errors, server health, database health, queue failures, disk space, suspicious authentication activity and backup completion.

Frequency	Recommended activities
Daily	Review critical errors, service availability, failed jobs and backup status
Weekly	Review security alerts, storage, database performance and forum reports
Monthly	Patch after testing, review roles, test restoration and assess vulnerabilities
Quarterly	Review architecture, access controls, incident response and disaster recovery

Maintain an incident response procedure: identify, contain, preserve relevant logs, eradicate the cause, recover safely and document lessons learned.

CHAPTER 14: IMPLEMENTATION ROADMAP

Phase	Deliverables
1. Discovery	Confirm pages, user journeys, roles, business rules and data requirements
2. Foundation	Laravel project, Vue project, MySQL schema, environments and CI checks
3. Public website	Layouts, home, about, services, news, FAQs and contact
4. Lead management	Consultation forms, inquiry workflow, email notifications and staff views
5. Member forum	Registration, login, categories, topics, replies and moderation
6. Administration	RBAC, content management, inquiry management and audit trail
7. Security hardening	Sanctum, CSRF, policies, throttling, headers, upload protection and secrets
8. Verification	Automated tests, security assessment, performance and accessibility
9. Deployment	Production configuration, monitoring, backups and recovery validation

14.1 Acceptance criteria

• Public pages load without authentication and expose only published content.

• Consultation and contact submissions are validated and protected against abuse.

• Forum write actions require authentication and server-side permission checks.

• Administrative endpoints reject unauthorized users.

• MySQL relationships and constraints protect data integrity.

• Security-sensitive actions are logged without recording secrets.

• Backups can be restored and production errors do not expose stack traces.

CHAPTER 15: CONCLUSION

The proposed architecture combines Vue.js for the presentation layer, Laravel for application processing and MySQL for relational data storage. Reusable frontend components, modular backend services, REST APIs, database constraints and layered security provide a maintainable foundation for a corporate website with consultation, contact, publishing, forum and administration capabilities.

The most important security principle is that the backend is the source of truth. Authentication, authorization, validation, data access and administrative permissions must be enforced in Laravel. Frontend restrictions improve user experience but do not provide security by themselves.

FINAL REVERSE-ENGINEERING NOTE

The component tree, database tables, endpoints, roles, infrastructure and security controls in this document are proposed design elements. They must be verified or adapted against the actual application before being described as existing implementation facts. Actual source-code analysis, database inspection and authorized testing are required to establish the live system's internal architecture.

REFERENCE TECHNOLOGY DOCUMENTATION

For implementation, consult the official documentation for the selected supported versions of Laravel, Laravel Sanctum, Vue.js, Vue Router, Pinia, MySQL, Nginx and PHP. Confirm version-specific configuration before deployment.